



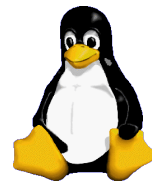
Administración básica de Linux

Copyright (c) 2005 Antonio de Andrés Lema, Vanesa Resúa Eiras e Carlos Carrión Álvarez.

Garántese permiso de copiar, distribuir e/ou modificar este documento baixo os termos da “GNU Free Documentation License”, versión 1.2 ou calquera versión publicada máis tarde pola Free Software Foundation; sen seccións invariantes, textos de portada, nen de contra portada. Unha copia de esta licencia se pode atopar en <http://www.gnu.org/licenses/fdl.txt>

Rógase notificar melloras, suxerencias e comentarios sobre este documento a aalema@edu.xunta.es

Configurando Linux: Os scripts



- Os comandos que xa coñecemos permítennos manexar e configurar moitos aspectos dun sistema Linux. Estes comandos poden ser executados de dous modos:
 - En **modo interactivo**, tecleando comandos nun terminal sobre o intérprete de comandos.
 - Mediante **ficheiros de comandos**, que automatizan unha tarefa xa que realizan dun golpe todos os comandos necesarios para ela.
- Estes ficheiros de comandos son coñecidos en Linux por **scripts** ou **shell scripts** (xa que son executados polo shell).
- Nun principio, podemos dicir que basicamente os scripts non son máis que una secuencia de comandos escritos nun ficheiro de texto.
- Sen embargo, os intérpretes de comandos de Linux inclúen toda unha serie de estruturas que dan lugar a un auténtico linguaxe de programación, con variables, instrucións e comandos de entrada/saída, estruturas de control, subprogramas, paso de parámetros, etc.

Scripts en Linux: Aspectos básicos



- O primeiro que deberemos introducir no script é unha liña na que se indique para que intérprete de comandos está programado. Hai que ter en conta que os distintos shells teñen funcións e estruturas de control distintas. A sintaxe será: `#!/ruta_shell`

`#!/bin/bash`

- A continuación introduciremos liñas con comandos e estruturas permitidas no shell que esteamos programando
- O carácter `#` nos permite introducir un comentario, xa que o shell ignorará todo o introducido a continuación
- Para poder executar o script deberemos darlle permisos de execución, utilizando o comando `chmod`
- A extesión habitual dun ficheiro de script en Linux é `.sh`, aínda que tamén é frecuente non poñerlles ningunha extesión.

Scripts en Linux: Definición de variables



- Podemos definir unha variable coa instrucción: ***variable=valor***
- Para acceder ó valor da variable introducimos o caracter \$ antes do nome da mesma: ***echo \$variable***
- Tamén podemos introducir o valor da variable dentro dunha cadea maior, tendo en conta que:
 - Se englobamos a cadea entre comillas dobres, o shell substituirá a variable polo seu valor.
 - Se englobamos a cadea entre comillas simples, o shell non fará ningunha substitución e mostrará o texto que se indica
 - Se englobamos a cadea entre comillas invertidas, o shell interpretará a cadea como un comando e devolverá o resultado (o mesmo se consegue con ***\$(comando)***):

```
var=mundo
echo "hola, $var" # mostra hola, mundo
echo 'hola, $var' # mostra hola, $var
echo `ls` # mostra o contido do directorio actual
```

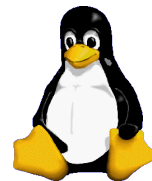
Scripts en Linux: Avaliação de expresiões



- Se queremos que bash avalie o contido dunha expresión aritmética e devolva o seu resultado usaremos a sintaxe **$\$((\text{expressión}))$** :

 $p = \$((p+3))$ #suma 3 á variable p
- Tamén podemos utilizar a sentencia **let**
- A sintaxe para avialiar expresiões condicionais (que devolven 1 ou 0) é **$[[\text{expressión}]]$** , onde a expresión pode ser:
 - Unha comparación con $=, !=, <, <=, >, >=$
 - A negación dunha expresión: $! \text{expressión}$
 - O and ou or de varias expresiões (deben encerrarse entre parénteses): $\text{expressión1} \&\& \text{expressión2}, \text{expressión1} || \text{expressión2}$

Scripts en Linux: Variables de contorno



- En principio, as variables que utilizemos dentro dun script só son accesibles dende o propio script.
- A orde **export** permítenos exportar unha variable ó **contorno do sistema**, de forma que quedará almacenada mentres dure a sesión e poderá ser accesible dende calquera outro proceso:

export var #exporta a variable var

export var2=mundo2 #asígnalle un valor á variable e a exporta

- O sistema ten predefinidas unha serie de **variables de contorno** que tamén se poden manipular dende os scripts:
 - **PS1**: Mensaxe do prompt (ver sintaxe no manual de bash)
 - **HOME**: Ruta do directorio home do usuario
 - **PATH**: Lista de rutas nas que se buscan os comandos no sistema, separadas por :
 - **SHELL**: Ruta do shell que estamos utilizando
 - **DISPLAY**: Consola pola que se redirixe a saída do sistema

Scripts en Linux: Paso de parámetros



- En moitos casos nos interesará que o script poida recibir parámetros na liña de comandos, e acceder a eles dende o script. Para facelo, utilizaremos a seguinte sintaxe:
 - **\$#** devolve o número de parámetros que se lle pasou ó script
 - **\$@** devolve a cadea con todos os parámetros
 - **\$0** devólvenos o nome do propio script que se está executando
 - **\$n** devolve o valor do parámetro n pasado ó script
 - **Shift [n]** é unha instrucción que despraza cara a “esquerda” todos os parámetros recibidos no script, é dicir, o \$2 pasa a ser o \$1, \$3 pasa a \$2, etc. Pódese indicar como parámetro de shift o número de posicións que nos queremos desprazar (por defecto 1).
- Tamén podemos ler parámetros da entrada estándar dentro do script coa función **read**:

read NOME # le da entrada estándar a variable NOME



Scripts en Linux: Control de fluxo

- Tamén podemos incluír dentro dos scripts as sentencias típicas de control de fluxo condicionais e iterativas de calquera linguaxe de programación

- **IF:**

- Sintaxe:

```
if [ condición1 ]; then acción1
else if [ condición2 ]; then acción2
    else acción
    fi
fi
```

A acción pode ser outro if, tendo así ifs anidados

Podemos substituír o “else if” por “elif”, eliminando así o “fi”

- **TEST:**

- Permite avaliar unha expresión e ver se é verdadeira ou falsa. Útilízase con frecuencia nas condicións do if
 - A expresión pode ser:



Scripts en Linux: Control de fluxo

-r ficheiro	o ficheiro existe e se pode ler
-w ficheiro	o ficheiro existe e se pode escribir
-x ficheiro	o ficheiro existe e se pode executar
-f ficheiro	o ficheiro existe e é un ficheiro normal
-d ficheiro	o ficheiro é un directorio
-c ficheiro	é un ficheiro especial de tipo carácter
-b ficheiro	é un ficheiro especial de tipo bloque
-s ficheiro	o ficheiro ten un tamaño maior que cero
-z s1	a lonxitude da cadea s1 é cero
-n s1	a lonxitude da cadea s1 non é cero
s1 = s2	as dúas cadeas son iguais
s1_ = s2	as dúas cadeas son distintas
s1	a cadea s1 existe
n1 -eq n2	n1 e n2 teñen o mesmo valor numérico
n1 -ne n2	n1 e n2 teñen distinto valor numérico
n1 -gt n2	n1 ten un valor maior que n2
n1 -lt n2	n1 ten un valor menor que n2
n1 -ge n2	n1 ten un valor maior ou igual que n2
n1 -le n2	n1 ten un valor menor ou igual que n2
-a	And lóxico
-o	Or lóxico



Scripts en Linux: Control de flujo

- **CASE:**

- Sintaxe:

```
case variable in
  patrón1) comandos a executar se variable coincide con patrón1;;
  patrón2) comandos a executar se variable coincide con patrón2;;
  ...
esac
```

- Cada patrón posible do case pode ser:

- Un valor constante, numérico ou de cadea
 - Un conxunto de valores constantes, separados por |
 - Un rango de valores, separando o mínimo e o máximo por -
 - Patrón por defecto, con *



Scripts en Linux: Control de flujo

- **Comando1 && Comando2:** Executa Comando2 só se Comando1 ten éxito (código de saída 0)
- **Comando1 || Comando2:** Executa Comando2 só se Comando1 non ten éxito (código de saída distinto de 0)
- **FOR:**
 - Sintaxe:
for variable in lista-valores
do
Instruccions a executar
done
 - Onde a lista de valores pode ter:
 - Un conxunto de valores separados por espazos
 - O resultado dun comando entre ``
 - O conxunto de parámetros do script (\$@)



Scripts en Linux: Control de flujo

- **WHILE/UNTIL:**

- Sintaxe:

```
while condición  
do  
Comandos a executar  
done
```

```
until condición  
do  
Comandos a executar  
done
```

Execútase ata que se
cumpre a condición

- **BREAK E CONTINUE:**

- Igual que en C, estas sentencias permiten saír do bucle e pasar á seguinte iteración respectivamente

- **EXIT:**

- Sae do script

- **SLEEP:**

- Fai unha pausa do número de segundos indicado

Scripts en Linux: Definición de funciones



- Unha función nos permite englobar un conxunto de comandos baixo un nome que podemos invocar dende o script
- Sintaxe:

```
function nome ()  
{  
    comandos a executar  
}
```
- A sentencia **return** permite saír da función
 - Podemos indicar opcionalmente o valor de retorno da función
- Para invocar a función, simplemente introduciremos o seu nome seguido dos posibles parámetros, que se recollerán na función da mesma forma que se recollen no script, como \$1, \$2, etc.